



Tema 11 — Consultas multitable y subconsultas



1. Introducción y contextualización práctica

- Permite trabajar con varias tablas relacionadas
 - Base de consultas reales en aplicaciones
-



2. Consultas sobre varias tablas

- Se usan varias tablas en un mismo SELECT
- Relación mediante:
 - Clave primaria (PK)
 - Clave externa (FK)

👉 Regla:
PK ↔ FK

💥 Base de casi todo el tema



3. Composiciones internas

- = INNER JOIN
- Devuelven SOLO coincidencias

```
SELECT *  
FROM empleados e  
JOIN departamentos d  
ON e.dep_id = d.id;
```

💥 JOIN = INNER JOIN

4. Producto cartesiano

- CROSS JOIN
- Combina todas las filas con todas

⚠ Genera muchas filas → cuidado

💥 Pregunta trampa típica

5. Composiciones externas

- LEFT JOIN → todo izquierda
- RIGHT JOIN → todo derecha
- FULL OUTER JOIN → todo ambas

💥 Clave examen:

Composiciones externas = LEFT + RIGHT + FULL

♦ 6. FULL OUTER JOIN

- Devuelve:
 - Coincidencias
 - No coincidencias de ambas tablas

★ Menos preguntado pero entra

7. Subconsultas

- Consulta dentro de otra consulta

Tipos:

- Escalar
- De fila
- De tabla

💥 Respuesta típica: todas

♦ 8. Subconsultas con ALL, ANY, IN, NOT IN

- IN → dentro
- NOT IN → fuera
- ANY → cumple alguno
- ALL → cumple todos

WHERE departamento NOT IN
(SELECT departamento FROM despidos);

💡 NOT IN → excluir resultados

♦ 9. Subconsultas avanzadas

EXISTS / NOT EXISTS

- EXISTS → existe resultado
 - NOT EXISTS → no existe
-

Uso con:

- FROM
- JOIN
- GROUP BY
- HAVING

★ Más teórico pero puede caer

10. Operaciones de conjuntos

INTERSECT

- Filas comunes
 - ❌ sin duplicados
-

MINUS (EXCEPT)

- Filas de la primera que no están en la segunda

💡 MINUS = quitar resultados

◆ EXTRA SCORM — NATURAL JOIN ★

- No usa **ON**
- Usa columnas con el mismo nombre

💡 Pregunta típica

! Trampas de examen

- JOIN = INNER JOIN
 - LEFT JOIN → no solo coincidencias
 - NATURAL JOIN → sin ON
 - INTERSECT → sin duplicados
 - MINUS = EXCEPT
 - CROSS JOIN → producto cartesiano
-

🧠 Resumen rápido

- PK ↔ FK
 - JOIN = INNER
 - LEFT → todo izquierda
 - CROSS → todo con todo
 - NOT IN → excluir
 - INTERSECT → común
 - MINUS → quitar
-

🚀 Clave final

Tema de **SQL avanzado puro**

Domina:

- JOINS
- Subconsultas
- Operaciones de conjuntos

✓ aprobado asegurado



Tema 12 — Opciones para el tratamiento de los datos



1. Introducción y contextualización práctica

- Permite manipular datos en BD
 - Base de aplicaciones reales
-



2. Inserción de registros

INSERT INTO tabla VALUES (...);



Comando clave: INSERT



3. Modificación de registros

UPDATE tabla
SET campo = valor
WHERE condicion;



UPDATE = modificar



4. Eliminación de registros

DELETE FROM tabla
WHERE condicion;

DELETE FROM empleados
WHERE num_departamento = 5;



Elimina empleados de ese departamento

5. Integridad referencial

- Relación entre tablas mediante FK

👉 Si valor FK:

- ❌ No existe en tabla referenciada

👉 Resultado:

ERROR → **no se realiza operación**

💥 Muy preguntado

6. Consultas al diccionario de datos

- Información interna de la BD

Ejemplos:

- ALL_TABLES
- ALL_TAB_COLUMNS

💥 Son vistas

7. Extensiones: XML

- Permite almacenar datos estructurados

Funciones:

- XMLTYPE → almacenar
- EXTRACT / EXTRACTVALUE → consultar

```
SELECT EXTRACTVALUE(columna_xml, '/persona/nombre')  
FROM tabla;
```

★ No salió en test pero puede caer

8. Extensiones: JSON

- Formato moderno de datos

JSON_TABLE

👉 Convierte:

- JSON → filas y columnas

💥 Pregunta de examen

9. PIVOT y UNPIVOT

- PIVOT → filas → columnas
- UNPIVOT → columnas → filas

💥 UNPIVOT muy preguntado

10. Agregaciones avanzadas

- ROLLUP → subtotales
- CUBE → combinaciones
- GROUPING SETS → agrupaciones personalizadas

💥 Pregunta típica

EXTRA — Variables enlazadas 💥

```
SELECT *  
FROM libros  
WHERE autor_id = :id_autor;
```

👉 :id_autor =
variable enlazada

Trampas de examen

- INSERT / UPDATE / DELETE → no confundir
 - FK inválida → ERROR
 - ALL_TABLES → vista
 - JSON_TABLE → JSON → tabla
 - UNPIVOT → columnas → filas
 - DELETE → elimina
 - :variable → bind variable
-

Resumen rápido

- INSERT → añadir
 - UPDATE → modificar
 - DELETE → eliminar
 - FK mal → error
 - JSON_TABLE → JSON a tabla
 - XML ★ → almacenar + consultar
 - UNPIVOT → columnas a filas
 - ROLLUP / CUBE → agregaciones
 - :id → variable
-

Clave final

Tema de:

👉 manipulación + extensiones SQL

Domina:

- INSERT / UPDATE / DELETE
- Integridad referencial
- JSON + ★ XML
- PIVOT / UNPIVOT
- ROLLUP / CUBE

✓ aprobado asegurado



Tema 13 — Elementos de explotación de tablas. Gestión de la seguridad



1. Introducción y contextualización práctica

- Gestión de:
 - Rendimiento (índices)
 - Acceso a datos (vistas)
 - Seguridad (usuarios, roles, políticas)

👉 Base de administración de BD



2. Concepto de índice

- Estructura que mejora la velocidad de búsqueda

👉 Función:

- Acceder más rápido a los datos

💥 Clave examen:

Índice = acelerar consultas



3. Creación de índices

```
CREATE INDEX idx_nombre  
ON tabla(columna);
```

💥 Se crean con:

```
CREATE INDEX
```

4. Eliminación de índices

DROP INDEX idx_nombre;

👉 Elimina el índice

5. Creación, manipulación y borrado de vistas

♦ Qué es una vista

- Consulta guardada como si fuera tabla
-

♦ Operaciones

- CREATE VIEW
 - ALTER VIEW
 - DROP VIEW
-

♦ Inserción en vistas

👉 Solo posible si:

- Vista simple
 - Una sola tabla
 - Sin filtros ni agrupaciones
-

6. Vistas de restricciones y sinónimos

♦ Restricciones

- ALL_CONSTRAINTS
- USER_CONSTRAINTS
- ALL_CONS_COLUMNS

 Son vistas del diccionario

♦ Sinónimos ★

- Alias para objetos de BD

👉 Facilitan acceso

👤 7. Gestión de usuarios y DCL

♦ Crear usuario

CREATE USER nombre;

💥 Muy preguntado

♦ DCL (Data Control Language)

- GRANT → dar permisos
- REVOKE → quitar permisos

💥 Comandos DCL = GRANT y REVOKE

🔑 8. Gestión de roles y privilegios

- Roles → conjunto de permisos
- Privilegios → permisos individuales

👉 Se asignan sobre:

- TABLE 💥
 - Otros objetos ★
-

📋 9. Perfiles y normativa de protección de datos ★

- Perfiles → controlan recursos y seguridad
- Ejemplo:
 - Límites de sesiones
 - Tiempo de conexión

👉 Relacionado con protección de datos



10. Seguridad de la información (políticas)

♦ Seguridad ante fallos lógicos 💥

- Gestión de usuarios
- Roles y perfiles

👉 Protege contra errores humanos

♦ Buenas prácticas 💥

- Control de accesos
- Restricciones de uso
- Políticas de software

👉 Respuesta típica: **todas las anteriores**



11. Políticas de bloqueo

- Bloqueo por intentos fallidos
- Restricción por horario
- Control de sesiones

💥 Respuesta típica: todas



Trampas de examen

- Índices → NO modifican datos
 - CREATE USER → correcto
 - GRANT / REVOKE → DCL
 - Vistas → no siempre insertables
 - Índices → acceso rápido
 - ALL_CONSTRAINTS → vista
 - TABLE → objeto típico de privilegios
-

Resumen rápido

- Índices → acelerar
 - CREATE INDEX / DROP INDEX
 - Vistas → consulta guardada
 - INSERT en vista → solo simples
 - DCL → GRANT / REVOKE
 - Usuario → CREATE USER
 - Roles → conjunto permisos
 - Seguridad → usuarios + políticas
-

Clave final

Tema de:

👉 rendimiento + seguridad

Domina:

- Índices
- Vistas
- Usuarios y permisos
- Políticas de seguridad

✓ aprobado asegurado



Tema 14 — Transacciones y problemas de concurrencia



1. Introducción y contextualización práctica

- Permite gestionar operaciones seguras en BD
- Evita errores cuando varios usuarios acceden a la vez

👉 Base de sistemas reales multiusuario



2. Transacciones

♦ Qué es

- Conjunto de operaciones que se ejecutan como una unidad
-

♦ Propiedades (ACID)

- Atomicidad → todo o nada
- Consistencia → estado correcto ✨
- Aislamiento → no interferencias ★
- Durabilidad → cambios permanentes ★

🌟 Muy preguntado:

Consistencia = BD estable tras transacción

♦ Inicio y fin

- Empieza con primera instrucción DML
- Termina con:

COMMIT;
ROLLBACK;

🌟 Pregunta típica



3. SAVEPOINT y estado de datos

♦ SAVEPOINT

SAVEPOINT punto1;

👉 Crea punto intermedio

♦ ROLLBACK TO SAVEPOINT

ROLLBACK TO punto1;

👉 Vuelve a ese punto

♦ RELEASE SAVEPOINT 💥

RELEASE SAVEPOINT punto1;

👉 Elimina el savepoint



4. Problemas de concurrencia

♦ Tipos

- Lectura sucia → leer datos no confirmados 💥
 - Lectura no repetible → cambia el dato
 - Lectura fantasma → aparecen nuevas filas 💥
 - Modificación perdida ★
-



5. Control de concurrencia

♦ Técnicas

- Bloqueos
- Marcas de tiempo
- Validación

💥 Respuesta: todas las anteriores

♦ Tipos de control

- Pesimista → bloquea antes
- Optimista → comprueba después

💥 Muy típico

6. Transacciones distribuidas

- Se ejecutan en varios nodos

♦ COMMIT de dos fases 💥

- Asegura coherencia en todos los nodos

👉 Resultado:

✓ Todos los nodos completan la transacción

7. Políticas de bloqueo

♦ Niveles de bloqueo 💥

- Tabla
- Fila
- Página

👉 Pregunta típica

♦ Objetivo

- Evitar conflictos entre usuarios
-

8. Niveles de procesamiento de consulta ★

♦ Procesadores locales

- Ejecutan en un solo sistema

♦ Procesadores distribuidos

- Ejecutan en varios nodos

👉 Relacionado con BD distribuidas

! Trampas de examen

- Consistencia ≠ durabilidad
 - COMMIT / ROLLBACK → fin transacción
 - Lectura fantasma → filas nuevas
 - Lectura sucia → datos no confirmados
 - Bloqueos → tabla, fila, página
 - RELEASE SAVEPOINT → eliminar punto
 - COMMIT 2 fases → todos los nodos
-

🧠 Resumen rápido

- Transacción → conjunto operaciones
 - ACID → Consistencia clave
 - COMMIT / ROLLBACK → final
 - SAVEPOINT → punto intermedio
 - Lectura sucia / fantasma → errores
 - Bloqueos → tabla, fila, página
 - Pesimista / optimista → control
 - 2 fases → nodos coordinados
-

🚀 Clave final

Tema de:

👉 control de datos concurrentes

Domina:

- ACID
- Problemas de concurrencia
- Bloqueos
- SAVEPOINT
- Transacciones distribuidas

✓ aprobado asegurado



Tema 15 — El lenguaje PL/SQL

1. Introducción y contextualización práctica

- PL/SQL = extensión de SQL con programación
- Permite:
 - Lógica
 - Control de flujo
 - Automatización

👉 Muy usado en Oracle

2. PL/SQL: Definición y reglas

- No distingue mayúsculas/minúsculas ★
- Las instrucciones terminan en ; ✨
- Comentarios:
 - `--` una línea
 - `/* */` varias líneas

👉 ✨ Pregunta típica: “todas son correctas”

3. Unidades léxicas

- Identificadores → nombres
- Literales → valores
- Delimitadores → separan sentencias ✨
- Comentarios

👉 ✨ Delimitadores = separar instrucciones

4. Tipos de datos simples

- Numéricos
- Carácter
- Booleanos
- Fecha
- LOB

✨ Respuesta correcta típica

5. Variables, constantes y operadores

◆ Declaración

DECLARE

variable NUMBER;

◆ Constantes

año CONSTANT NUMBER := 1985;

💥 Diferencia clave:

- Variable → cambia
 - Constante → no cambia
-

◆ Operadores

- Aritméticos (+, -, *, /, **)
- Lógicos (AND, OR)

6. Condicionales

IF condicion THEN

...

ELSIF condicion THEN

...

ELSE

...

END IF;

💥 Ejecuta según condición

7. Bucles

♦ WHILE

WHILE condicion LOOP

...

END LOOP;

👉 Si condición no se cumple:

✓ no ejecuta nada 💥

♦ FOR ★

- Itera número de veces definido
-

! 8. Manejo de errores: Excepciones

👉 Sirven para:

✓ manejar errores de forma controlada 💥

♦ Tipos

- Definidas por usuario
- Predefinidas por Oracle
- No definidas

💥 Clasificación completa

9. Librerías y funciones

- Funciones matemáticas
- Funciones de agregación

💥 Muy preguntado

10. Entornos de desarrollo

- MySQL Workbench
- SQL Server Management Studio
- Oracle SQL Developer

 Pregunta típica

11. Tipos de scripts

♦ Scripts simples ★

- Ejecutan instrucciones secuenciales

 Base para automatización

Trampas de examen

- PL/SQL → no distingue mayúsculas
 - ; obligatorio
 - WHILE puede no ejecutar
 - Excepciones → manejar errores
 - Delimitadores → separar
 - Tipos → incluyen LOB
 - IF → no se ejecuta siempre
-

Resumen rápido

- PL/SQL → SQL + programación
- ; → fin instrucción
- Tipos → numérico, char, booleano, fecha, LOB
- IF → condicional
- WHILE → puede no ejecutar
- Excepciones → errores
- Librerías → matemáticas/agregación



Tema 16 — Tipos de datos compuestos y cursores

1. Introducción y contextualización práctica

En PL/SQL no solo trabajamos con variables simples (números, texto...), sino con **estructuras más complejas**:

- Registros → agrupan datos relacionados
- Arrays → colecciones de valores
- Cursores → permiten recorrer resultados de consultas

👉 Esto es clave cuando trabajas con **múltiples filas o estructuras complejas**

2. Registros

♦ Qué es

Un **registro** es una estructura que agrupa varios campos relacionados (como una fila de una tabla)

👉 Ejemplo:

```
TYPE persona IS RECORD (  
    nombre VARCHAR2(50),  
    edad NUMBER  
);
```

💥 Clave examen:

Registro = conjunto de campos relacionados

♦ %ROWTYPE 💥

Permite crear un registro con la **misma estructura que una fila de una tabla**

```
empleado empleados%ROWTYPE;
```



3. Uso y asignación de registros

- Se pueden asignar valores campo a campo
- O copiar registros completos

👉 Ejemplo conceptual:

```
persona.nombre := 'Pau';
```

👉 Muy útil para trabajar con datos de tablas



4. Arrays: Definición y sintaxis

♦ Qué es un array

- Colección de elementos del mismo tipo

💥 Clave examen:

Todos los elementos son del mismo tipo

♦ Características

- Se crean vacíos ★
 - Pueden crecer dinámicamente ★
-



5. Arrays: asignación y métodos

♦ Métodos importantes

- `.FIRST` → primer elemento
- `.LAST` → último elemento 💥
- `.COUNT` → número de elementos ★

👉 Ejemplo:

```
array_z.LAST;
```

✓ Devuelve el último elemento

6. Arrays asociativos

- No tienen tamaño fijo
- Acceso por clave (tipo diccionario)

💥 Clave:

Muy rápidos para búsquedas

7. Tablas anidadas

- Tipo de array más estructurado
- Se pueden almacenar en BD ★

👉 Diferencia:

- Más “formales” que arrays asociativos
-

8. Cursores

◆ Qué es

- Mecanismo para recorrer resultados de consultas

👉 Cuando una consulta devuelve varias filas

◆ Funcionamiento básico

1. OPEN → abrir cursor
 2. FETCH → obtener fila 💥
 3. CLOSE → cerrar cursor
-

◆ Propiedades importantes 💥

- %FOUND → TRUE si el FETCH devuelve fila
- %NOTFOUND → no hay más filas
- %ISOPEN → cursor abierto
- %ROWCOUNT → filas procesadas ★

◆ **FETCH** 🌟

FETCH cursor INTO variable;

👉 Extrae datos del cursor

🔄 **9. Tipos de cursores**

◆ **Implícitos** ★

- Automáticos (los crea el sistema)

◆ **Explícitos** 🌟

- Los define el programador
 - Se gestionan manualmente
-

🔄 **10. Ciclo de vida de cursores explícitos**

1. Declarar
2. Abrir (OPEN)
3. Leer (FETCH)
4. Cerrar (CLOSE)

🌟 Orden típico de examen

🔄 **11. Actualización con cursores** ★

- Permiten modificar datos mientras recorres resultados

👉 Uso más avanzado

👉 Puede caer conceptual

Trampas de examen

- Registro ≠ array
 - %ROWTYPE → estructura completa
 - Arrays → mismo tipo
 - .LAST → último elemento
 - %FOUND → TRUE si hay fila
 - %ISOPEN → cursor abierto
 - FETCH → obtener datos
 - Cursores explícitos → los crea el usuario
-

Resumen rápido

- Registro → varios campos
 - %ROWTYPE → copia estructura tabla
 - Array → colección
 - .LAST → último
 - Cursores → recorrer resultados
 - FETCH → obtener fila
 - %FOUND → hay resultado
-

Clave final

Tema de:

 estructuras complejas + acceso a datos

Domina:

- Registros
- Arrays
- Cursores

✓ aprobado asegurado



Tema 17 — Subprogramas y disparadores

1. Introducción y contextualización práctica

En PL/SQL podemos crear **bloques reutilizables de código** y automatizar acciones:

- Subprogramas → reutilización (procedimientos, funciones)
- Disparadores → automatización automática

👉 Clave en aplicaciones reales (lógica en BD)

2. Procedimientos

♦ Qué es

- Bloque de código con nombre
- Realiza una acción concreta

💥 Clave examen:

Procedimiento = código reutilizable con un propósito

♦ Características

- Puede recibir parámetros
 - No devuelve valor (directamente)
-

♦ Uso

`procedimiento_nombre(parametros);`

💥 Se invoca por su nombre

3. Funciones

♦ Qué es

- Similar a procedimiento PERO:
 - ✓ devuelve un valor
-

♦ Diferencia clave

- Procedimiento → no devuelve valor
 - Función → sí devuelve
-

♦ Uso típico

- Dentro de consultas SQL
-

4. Paquetes

♦ Qué son

- Agrupan elementos relacionados

 Incluyen:

- Funciones
 - Procedimientos
 - Tipos de datos
-

♦ Ventajas

- Organización
 - Reutilización
 - Mantenimiento
-

⚡ 5. Disparadores (Triggers)

♦ Qué son

- Código que se ejecuta automáticamente ante eventos

👉 Eventos típicos:

- INSERT
 - UPDATE
 - DELETE
-

♦ Para qué sirven 🌟

- Controlar cambios
- Automatizar tareas
- Aplicar reglas

👉 Respuesta típica: **todas correctas**

⚙️ 6. Explotación: restricciones, orden y gestión

♦ Uso

- Validar datos
 - Forzar reglas de negocio
-

♦ Orden

- BEFORE → antes
- AFTER → después

★ Puede caer conceptual

♦ Restricción importante 🌟

- No se deben usar:
 - COMMIT
 - ROLLBACK
 - SAVEPOINT

👉 Dentro de triggers están **prohibidos**

7. Referencias y condicionales

♦ IF INSERTING / UPDATING / DELETING

Permiten:

👉 Saber qué operación activó el trigger

♦ Referencias

- :OLD → valor anterior
- :NEW → valor nuevo

★ Muy típico en teoría

8. Tipos especiales de triggers

♦ Tipos

- A nivel fila 
→ se ejecuta por cada fila
 - A nivel sentencia ★
→ se ejecuta una vez
-

♦ INSTEAD OF

- Se usan en vistas

👉 Permiten ejecutar lógica en lugar de la operación

9. Automatización de scripts

- Permite ejecutar tareas automáticamente

👉 Ejemplos:

- Limpieza de datos
 - Backups
-

10. Oracle Scheduler

- Herramienta para programar tareas

 Muy importante

♦ Alternativas

- DBMS_JOB
 - Enterprise Manager
-

11. API de acceso a bases de datos

- Permiten conectar aplicaciones con BD

 Ejemplos:

- JDBC
 - ODBC
-

Trampas de examen

- Procedimiento ≠ función
- Procedimiento → no devuelve valor
- Trigger fila → por cada fila
- COMMIT → prohibido en triggers
- INSTEAD OF → en vistas
- IF INSERTING → identifica operación
- Paquete → agrupa funciones/procedimientos

Resumen rápido

- Procedimiento → acción reutilizable
- Función → devuelve valor
- Paquete → agrupa código
- Trigger → automático
- Fila → se ejecuta varias veces
- INSTEAD OF → vistas
- Scheduler → automatización



Tema 18 — Bases de datos objeto-relacionales

1. Introducción y contextualización práctica

Las bases de datos objeto-relacionales combinan:

- Modelo relacional (tablas)
- Modelo orientado a objetos (clases, métodos)

👉 Permiten trabajar con datos más complejos

💡 Clave:

Integran SQL con conceptos de programación orientada a objetos

2. Definición y características

♦ Qué son

- BD que permiten definir tipos complejos (objetos)
-

♦ Características

- Tipos de datos avanzados
- Métodos asociados a datos
- Integración con lenguajes

💡 Pregunta típica:

👉 No tienen solo tipos simples

3. Tipos de datos objeto

♦ Qué es

- Representación de un objeto del mundo real

👉 Incluye:

- Atributos
- Métodos

💥 Clave:

No es solo nombre + atributos

4. Definición de tipos de datos objeto

♦ Sintaxis 💥

```
CREATE TYPE nombre_tipo AS OBJECT (  
    atributos  
);
```

👉 Muy preguntado

♦ Estructura ★

- Especificación (definición)
 - Cuerpo (implementación)
-

5. Declaración de atributos

♦ Características

- Pueden ser de cualquier tipo
- Pueden ser otros objetos 💥
- No llevan restricciones como NOT NULL 💥

👉 Se declaran antes de los métodos

6. Declaración de métodos

♦ Qué son

- Funciones o procedimientos del objeto
-

♦ Declaración

- Se usa **MEMBER**

MEMBER PROCEDURE metodo(...);

♦ Dónde

- Se definen en el cuerpo del objeto
-

7. Métodos estáticos

♦ Qué son

- Métodos que no dependen de una instancia

 Clave:

Se llaman usando el nombre del tipo

♦ Características

- No acceden a atributos de instancia

 Trampa típica

8. Parámetro SELF

♦ Qué es

- Referencia al propio objeto
-

♦ Valor por defecto

- IN

 Muy preguntado

9. Sobrecarga de métodos

♦ Qué es

- Mismo nombre, distintos parámetros

 Ejemplo:

metodo(x);

metodo(x, y);

10. Aplicación avanzada de sobrecarga

- Permite flexibilidad
- Mejora reutilización

 Puede caer conceptual

11. Constructores

♦ Qué son

- Métodos que crean objetos
-

♦ Función

- Inicializar atributos

★ Menos preguntado pero importante

Trampas de examen

- BD objeto-relacional ≠ solo relacional
 - CREATE TYPE → sintaxis exacta
 - Métodos estáticos → no usan instancia
 - SELF → IN
 - Sobrecarga → mismo nombre, distintos parámetros
 - Atributos → pueden ser objetos
 - NOT NULL → no permitido
-

Resumen rápido

- BD OR → mezcla relacional + objetos
 - CREATE TYPE → definir objeto
 - Atributos → datos
 - Métodos → lógica
 - STATIC → sin instancia
 - SELF → referencia objeto
 - Sobrecarga → mismo nombre
-



Tema 19 — Utilización de los objetos



1. Introducción y contextualización práctica

En las bases de datos objeto-relacionales podemos:

- Crear objetos
- Instanciarlos
- Acceder a atributos
- Utilizar herencia y polimorfismo



Acerca las BD a la programación orientada a objetos



2. Declaración de objetos

♦ Qué es

- Crear una variable de tipo objeto



Ejemplo:

```
alumno1 alumno;
```



Clave examen:

```
variable_objeto tipo_objeto;
```



3. Instancias de objetos

♦ Qué es instanciar

- Crear un objeto real a partir del tipo definido
-

♦ Sintaxis

```
venta5 := NEW venta (...);
```

 **NEW** crea/inicializa el objeto

 Muy preguntado

4. Referencias a atributos

♦ Acceso a atributos

```
profesor1.nombre := 'Ramiro';
```

 Se usa:

```
objeto.atributo
```

 Pregunta típica

5. Llamadas a métodos

♦ Invocación

```
profesor1.setSalario();
```

 Los métodos se llaman sobre el objeto

 Muy preguntado



6. Herencia

♦ Qué es

- Un subtipo hereda atributos y métodos del supertipo

👉 Igual que en POO clásica

♦ FINAL 🌟

Impide herencia o sobrescritura

FINAL

👉 Muy típico de examen

♦ INSTANTIABLE / NO INSTANTIABLE ★

- INSTANTIABLE → permite crear instancias
- NO INSTANTIABLE → no permite instancias directas

🌟 Puede caer perfectamente



7. Polimorfismo

♦ Qué es

- Un mismo método puede comportarse distinto según el objeto

👉 Base de reutilización avanzada

★ Muy conceptual



8. Método MAP

♦ Qué es

- Método especial para comparar objetos

♦ Restricción 🌟

- Solo puede existir UN método MAP por tipo de objeto

👉 Pregunta típica

🔄 9. Método ORDER ★

♦ Qué hace

- Permite comparar objetos directamente

👉 Similar a MAP pero más complejo

⚠ Puede confundirse con MAP

🔑 10. FINAL, INSTANTIABLE, NO INSTANTIABLE y SELF AS RESULT

♦ FINAL 🌟

- Impide sobrescritura/herencia
-

♦ INSTANTIABLE 🌟

- Permite crear instancias
-

♦ NO INSTANTIABLE ★

- Tipo abstracto (no instanciable)
-

♦ SELF AS RESULT ★

- Devuelve el propio objeto

👉 Relacionado con métodos constructores

Trampas de examen

- `NEW` → crear objeto
 - `objeto.atributo` → acceder atributo
 - `objeto.metodo()` → llamar método
 - `STATIC` → sobre el tipo, no instancia
 - `FINAL` → impide herencia
 - `MAP` → solo uno
 - `INSTANTIABLE` → permite instancias
-

Resumen rápido

- Objeto → instancia de tipo
 - `NEW` → crear objeto
 - `objeto.atributo` → acceso
 - `objeto.metodo()` → llamada
 - Herencia → reutilización
 - `MAP / ORDER` → comparación
 - `FINAL` → bloquear herencia
-

Clave final

Tema de:

 uso práctico de objetos en BD

Domina:

- Instancias
- Métodos
- Herencia
- `MAP / ORDER`

✓ aprobado asegurado



Tema 20 — Organización de los tipos de datos objeto



1. Introducción y contextualización práctica

En las bases de datos objeto-relacionales no solo podemos definir objetos, sino también:

- Almacenarlos
- Relacionarlos
- Consultarlos
- Modificarlos

👉 Esto permite trabajar con estructuras complejas directamente en la BD.



2. Tipos de datos colección

♦ Qué son

Son estructuras que almacenan varios elementos del mismo tipo.

👉 Tipos principales:

- VARRAY
- Tablas anidadas

♦ VARRAY

Características

- Elementos ordenados mediante índice 🌟
- Tamaño máximo definido previamente 🌟
- Acceso por posición

👉 Ejemplo conceptual:

VARRAY(10) OF NUMBER

⚠ Trampa:

- Sí tienen límite máximo
- Sí mantienen orden

♦ Tablas anidadas

Qué son

Colecciones similares a tablas internas.

```
TYPE id_departamento IS TABLE OF NUMBER;
```

💥 Esto declara una tabla anidada.

♦ Diferencia importante 💥

VARRAY

Tabla anidada

Tamaño máximo fijo

Tamaño dinámico

Mantiene orden

Orden no
garantizado

🏗️ 3. Tablas de objetos

♦ Qué son

Tablas cuyos registros son objetos completos.

```
CREATE TABLE t_alumnos OF alumno;
```

💥 Muy preguntado.

👉 Aquí:

- `alumno` = tipo objeto
 - `t_alumnos` = tabla de objetos
-

4. Tablas con columnas tipo objeto

♦ Qué son

Tablas normales que contienen columnas de tipo objeto.

```
CREATE TABLE alumno2 (  
    alumno1 alumno  
);
```

💥 Diferencia importante:

- Tabla de objetos → toda la fila es objeto
 - Columna objeto → solo una columna es objeto
-

5. Consultas sobre objetos

♦ Acceso a atributos

objeto.atributo

👉 Igual que en programación orientada a objetos.

♦ Consultas SELECT ★

Se pueden consultar:

- Objetos completos
 - Atributos concretos
 - Referencias
-

+ 6. Inserción de objetos

♦ Uso de NEW 🌟

venta5 := NEW venta (...);

👉 Crea e inicializa una instancia.

♦ INSERT sobre tablas objeto

INSERT INTO tabla VALUES (...);

👉 Se insertan objetos completos.

♦ REF 🌟

REF(p)

👉 Obtiene referencia a un objeto.

♦ Ejemplo importante 🌟

INSERT INTO cursos (id, nombre_curso, profesor_ref)

SELECT 2, 'Tecnología', REF(p)

FROM profesores p;

👉 Inserta:

- Datos del curso
 - Referencia al objeto profesor
-

7. Modificación de objetos

♦ UPDATE sobre atributos

UPDATE alumno3 a

SET a.c2.nombre = 'Federico'

WHERE a.c2.id_alumno = '876853';

💥 Modifica atributos internos del objeto.

✗ 8. Eliminación de objetos

♦ DELETE

DELETE FROM tabla

WHERE condicion;

👉 Elimina objetos almacenados.

9. Cláusula VALUE

♦ Qué hace

Permite trabajar con el objeto completo.

👉 Muy usada en:

- SELECT 💥

SELECT VALUE(a)

FROM alumnos a;

♦ Uso conceptual

- Comparar objetos
- Obtener objeto completo

⚠ Trampa:

- No se usa para UPDATE o DELETE normalmente en test.
-

10. Cláusula REF

♦ Qué hace

Obtiene referencias a objetos.

REF(objeto)

💥 Muy preguntado.

♦ Utilidad

- Relacionar objetos
 - Crear referencias entre tablas
-

Trampas de examen

- VARRAY → sí tiene tamaño máximo
- Tabla anidada → dinámica
- `CREATE TABLE ... OF alumno` → tabla
- `NEW` → crear objeto
- `REF` → referencia objeto
- `VALUE` → objeto completo
- UPDATE puede modificar atributos internos

Resumen rápido

- Colecciones → VARRAY y tablas anidadas
- VARRAY → ordenado y limitado
- Tabla objeto → fila = objeto
- Columna objeto → objeto dentro tabla
- REF → referencia
- VALUE → objeto completo
- NEW → crear instancia



SUPERANEXO FINAL — TEMAS 11–20 (EXAMEN TEST)



SQL MULTITABLA Y SUBCONSULTAS

♦ JOIN = INNER JOIN

- Solo devuelve coincidencias

♦ LEFT JOIN

- Todo izquierda + coincidencias derecha

♦ INTERSECT

- Filas comunes
-  Sin duplicados

♦ MINUS

- Filas de la primera que no están en la segunda

♦ NOT IN

- Excluir resultados
-



MANIPULACIÓN DE DATOS

♦ INSERT

- Añadir registros

♦ UPDATE

UPDATE empleados
SET departamento = 14
WHERE departamento = 77;

👉 Reasigna empleados del 77 → 14

♦ DELETE

- Eliminar registros

♦ **:id_autor** 🌟

:id_autor

👉 Variable enlazada (bind variable)

🌟 **SEGURIDAD E ÍNDICES**

♦ **Seguridad lógica**

- Usuarios
- Roles
- Perfiles

♦ **Índices**

- Aceleran consultas
- ❌ No modifican datos

♦ **UNIQUE** 🌟

- No permite duplicados

🌟 **TRANSACCIONES**

♦ **Propiedades ACID** 🌟

- Atomicidad
- Consistencia
- Aislamiento
- Durabilidad

♦ **Bloqueos** 🌟

- Tabla
- Fila
- Página

♦ **Lectura fantasma**

- Aparecen nuevas filas

♦ **COMMIT / ROLLBACK**

- Finalizan transacción
-

PL/SQL

♦ **Librerías importantes**

- Matemáticas
- Agregación

♦ **Registros**

- Conjunto de campos relacionados

♦ **%ROWTYPE**

- Copia estructura completa tabla

♦ **Cursores**

- FETCH → obtener fila
-

SUBPROGRAMAS Y TRIGGERS

♦ **Procedimiento**

- Código reutilizable

♦ **Función**

- Devuelve valor

♦ **Paquetes**

Agrupar:

- Funciones
- Procedimientos
- Tipos

♦ Triggers fila

- Se ejecutan por cada fila

♦ COMMIT en trigger ⚠

- Prohibido

♦ IF INSERTING / UPDATING

- Detecta operación DML
-

OBJETO-RELACIONAL

♦ CREATE TYPE

CREATE TYPE nombre AS OBJECT

♦ Métodos STATIC ⚡

- Se llaman sobre el tipo
- ❌ No sobre instancia

♦ SELF ⚡

- Valor por defecto → IN

♦ FINAL ⚡

- Impide sobrescritura/herencia

♦ Sobrecarga

- Mismo método
- Distintos parámetros

♦ Constructor

- Crear/inicializar objetos
-

UTILIZACIÓN DE OBJETOS

♦ **NEW**

NEW venta(...)

 Crear instancia

♦ **Acceso atributo**

objeto.atributo

♦ **Método**

objeto.metodo();

♦ **MAP**

- Solo uno por objeto

♦ **ORDER**

- Compara objetos

♦ **INSTANTIABLE**

- Permite instancias
-

ORGANIZACIÓN DE OBJETOS

♦ **VARRAY**

- Ordenado
- Tamaño máximo fijo

♦ **Tabla anidada**

- Tamaño dinámico

♦ **Tabla de objetos** 🌟

CREATE TABLE tabla OF objeto;

♦ **Tabla con columna objeto** 🌟

CREATE TABLE tabla (
columna objeto
);

♦ **REF** 🌟

- Referencia objeto

♦ **VALUE** 🌟

- Obtener objeto completo
- Muy usado en SELECT

♦ **UPDATE objetos** 🌟

- Modifica atributos internos

! TRAMPAS GLOBALES

- JOIN = INNER JOIN
- INTERSECT → sin duplicados
- STATIC → tipo, no instancia
- FINAL → no sobrescribir
- SELF → IN
- VARRAY → sí tiene límite
- Trigger fila → por cada fila
- FETCH → obtener fila
- REF ≠ VALUE
- UNIQUE → no duplicados

🧠 RESUMEN ULTRA RÁPIDO

SQL

JOIN, INTERSECT, UPDATE

Transacciones

ACID, bloqueos, COMMIT

PL/SQL

Registros, cursores, FETCH

Triggers

Automáticos, COMMIT prohibido

Objetos

NEW, STATIC, FINAL, MAP

Colecciones

VARRAY, tablas anidadas

REF / VALUE

Referencia vs objeto completo



LO QUE MÁS CAE (TOP)

1. JOIN / INTERSECT
2. ACID
3. FETCH / cursores
4. STATIC / FINAL / SELF
5. NEW / REF / VALUE
6. VARRAY vs tabla anidada
7. Triggers fila
8. UPDATE / INSERT / DELETE